

Spring Boot Library Management — Project Submission

1. Entity Relationship Design

The application manages two entities: **Author** and **Book**.

Entities

Entity	Attributes	JPA Annotations
Author	id (PK), name	@Entity, @Id, @GeneratedValue, @OneToMany
Book	id (PK), title, genre, author (FK)	@Entity, @Id, @GeneratedValue, @ManyToOne, @JoinColumn

Relationship

Author (1) ————— (*) Book One Author can write many Books. Each Book belongs to exactly one Author.

```
JPA Mapping: ``java // In Author.java @OneToMany(mappedBy = "author", cascade = CascadeType.ALL) private List books = new ArrayList<>();

// In Book.java @ManyToOne @JoinColumn(name = "author_id", nullable = false) private Author author; ``
```

2. Implementation Details

2.1 Populate Database (DataInitializer)

On application startup, `DataInitializer` (implements `CommandLineRunner`) seeds the database with 10 Authors and 10 Books if the author table is empty.

```
java @Component public class DataInitializer implements CommandLineRunner {
@Override public void run(String... args) { if (authorRepository.count() == 0) {
```

```
for (int i = 1; i <= 10; i++) { Author a = authorRepository.save(new
Author("Author " + i)); bookRepository.save(new Book("Book Title " + i, genre,
a)); } } } }
```

Result: 10 rows in AUTHOR table, 10 rows in BOOK table on first run.

2.2 Create Operation

Controller:

```
java @PostMapping("/library/add") public String addBook(@RequestParam
String title, @RequestParam String genre, @RequestParam Long authorId, Model
model) { try { libraryService.addBook(title, genre, authorId); return
"redirect:/library"; } catch (DataIntegrityViolationException e) {
model.addAttribute("error", "Data integrity violation: " + e.getMessage()); return
"add"; } }
```

JSP Form (add.jsp): Contains `<input>` for title and genre, `<select>` dropdown populated with all authors via JSTL `<c:forEach>` .

[!NOTE] Exception handling catches `DataIntegrityViolationException` (e.g., null author FK) and redisplay the form with an error message — the user never sees a blank error page.

2.3 Read Operation

Custom INNER JOIN Query (Repository layer):

```
java // BookRepository.java
@Query("SELECT new com.example.library.dto.BookAuthorDTO(b.id, b.title, b.genre,
a.name) " + "FROM Book b INNER JOIN b.author a") List<BookAuthorDTO>
findBooksWithAuthors();
```

DTO:

```
java public class BookAuthorDTO { private Long bookId; private String title;
private String genre; private String authorName; // constructor + getters }
```

Controller:

```
java @GetMapping("/library") public String listBooks(Model model) {
model.addAttribute("books", libraryService.getBooksWithAuthors()); return "list";
}
```

JSP (list.jsp): Renders a styled HTML table using JSTL `<c:forEach var="book"
items="${books}">` with columns: ID, Title, Genre, Author, Action (Edit button).

2.4 Update Operation

Controller (GET — show pre-filled form): `java @GetMapping("/library/update/{id}")`
`public String showUpdateForm(@PathVariable Long id, Model model) { Book book =`
`libraryService.getBookById(id); model.addAttribute("book", book);`
`model.addAttribute("authors", libraryService.getAllAuthors()); return "update"; }`

Controller (POST — save changes): `java @PostMapping("/library/update/{id}") public`
`String updateBook(@PathVariable Long id, @RequestParam String title, @RequestParam`
`String genre, @RequestParam Long authorId, Model model) {`
`libraryService.updateBook(id, title, genre, authorId); return "redirect:/library";`
`}`

JSP (update.jsp): Pre-fills Title and Genre fields using EL (`${book.title}` , `${book.genre}`).
 Author dropdown pre-selects the current author using `${author.id == book.author.id ?`
`'selected' : ''}`.

3. Screenshots

READ — Book List Page (/library)

Shows all 10 seeded books (+ any added) with Author name via INNER JOIN, and Edit buttons.

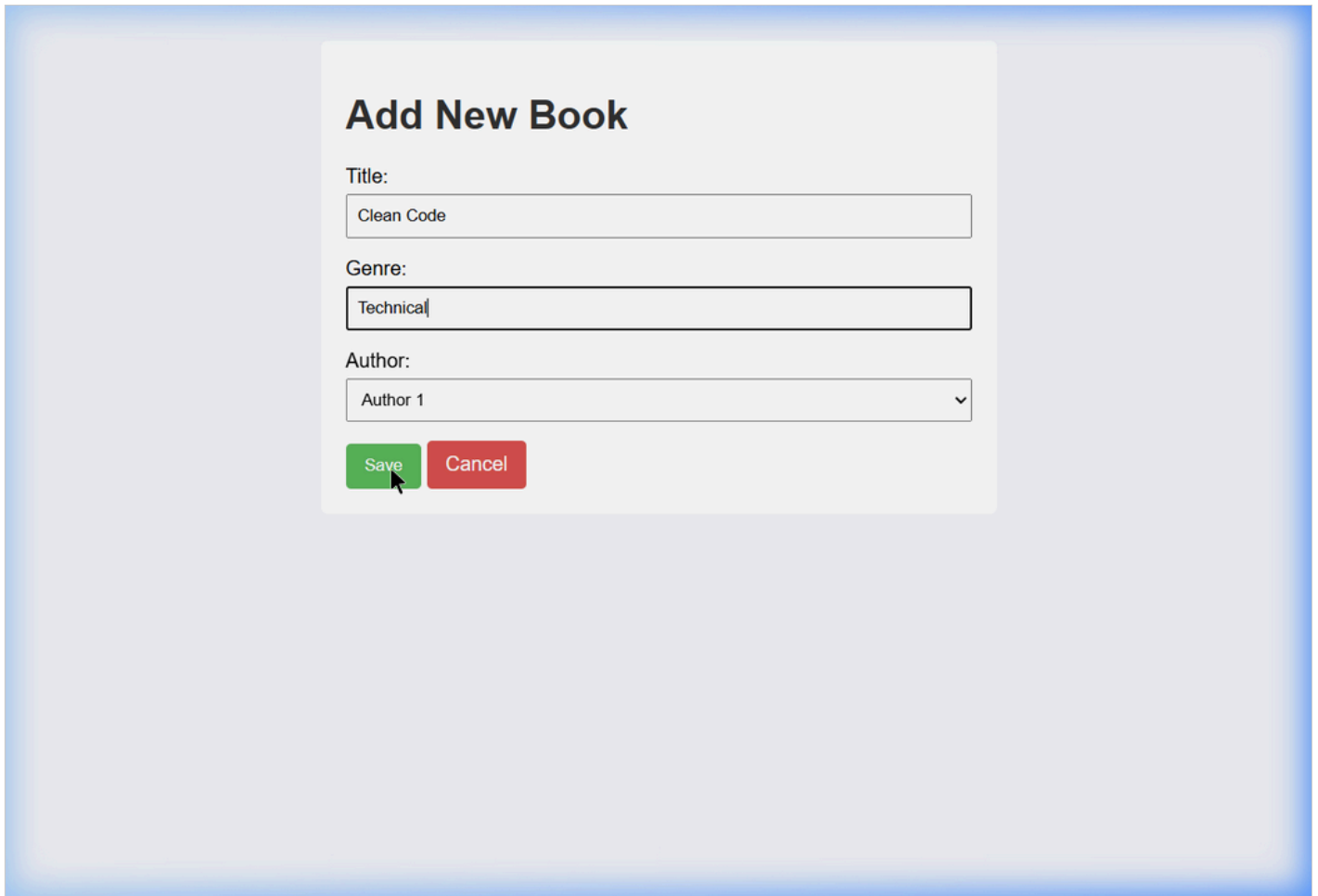
Library Management - Books

[Add New Book](#)

ID	Title	Genre	Author	Action
1	Book Title 1	Mystery	Author 1	Edit
2	Book Title 2	Genre 3	Author 2	Edit
3	Book Title 3	Genre 1	Author 3	Edit
4	Book Title 4	Genre 2	Author 4	Edit
5	Book Title 5	Genre 3	Author 5	Edit
6	Book Title 6	Genre 1	Author 6	Edit
7	Book Title 7	Genre 2	Author 7	Edit
8	Book Title 8	Genre 3	Author 8	Edit
9	Book Title 9	Genre 1	Author 9	Edit
10	Book Title 10	Genre 2	Author 10	Edit
11	Test Book	Science Fiction	Author 1	Edit
12	Clean Code	Technical	Author 1	Edit

CREATE — Add New Book (/library/add)

Form with Title, Genre text inputs and Author dropdown. Save button submits via POST.



Add New Book

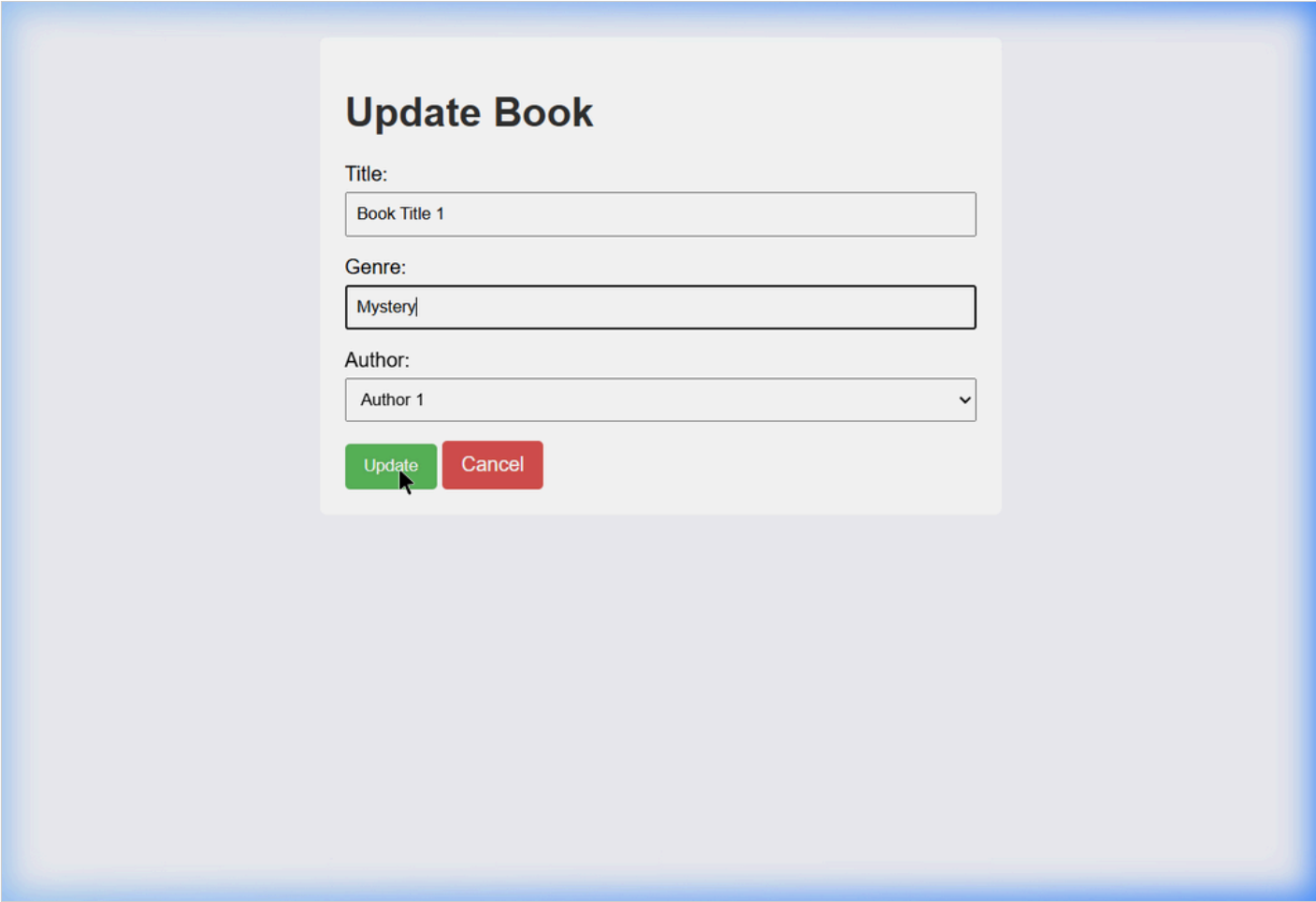
Title:

Genre:

Author:

UPDATE — Edit Book (/library/update/{id})

Pre-filled form with existing values. Genre changed to "Mystery" and submitted successfully.



4. Unit Tests

All **3 tests pass** (`mvnw test` → `BUILD SUCCESS`).

Test Class	Framework	What It Tests
BookRepositoryTest	@DataJpaTest + JUnit 5	Custom <code>findBooksWithAuthors()</code> INNER JOIN query
LibraryServiceTest	Mockito + JUnit 5	<code>addBook()</code> business logic with mocked repositories
LibraryApplicationTests	Spring Boot Test	Spring context loads without errors

Sample — Service Test (Mockito): ```java @Test void testAddBook() {
when(authorRepository.findById(1L)).thenReturn(Optional.of(author));
when(bookRepository.save(any(Book.class))).thenReturn(book);`

```

Book saved = libraryService.addBook("Test Book", "Fiction", 1L);

assertNotNull(saved);
assertEquals("Test Book", saved.getTitle());
assertEquals("John Doe", saved.getAuthor().getName());

}

```

Sample — Repository Test (DataJpaTest): ```java @Test void testFindBooksWithAuthors() { Author author = authorRepository.save(new Author("Jane Austen")); bookRepository.save(new Book("Pride and Prejudice", "Romance", author));`

```

List<BookAuthorDTO> result = bookRepository.findBooksWithAuthors();

assertFalse(result.isEmpty());
assertEquals("Pride and Prejudice", result.get(0).getTitle());
assertEquals("Jane Austen", result.get(0).getAuthorName());

}

```

5. Challenges Faced & Solutions

Challenge	How It Was Solved
JSP not rendering in Spring Boot 3.x	Added <code>tomcat-embed-jasper</code> and <code>jakarta.servlet.jsp.jstl</code> (Jakarta EE) dependencies to <code>pom.xml</code> . Old <code>javax.*</code> versions are incompatible with Spring Boot 3.
INNER JOIN returning entity fields, not strings	Created <code>BookAuthorDTO</code> with a JPQL constructor expression: <code>SELECT new com.example.library.dto.BookAuthorDTO(...)</code> . This lets JPA project query results directly into a DTO.
H2 Dialect warning	Removed the explicit <code>spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.H2Dialect</code> from <code>application.properties</code> — Hibernate 6 auto-detects it.

Challenge	How It Was Solved
Root URL 404	Added <code>@GetMapping("/")</code> that returns <code>redirect:/library</code> so users landing on the root are forwarded automatically.

6. Project Structure Summary

```
library-management/ ├── pom.xml ├── README.md ├── docs/screenshots/ ←
book_list.png, add_form.png, update_form.png └── src/ ├── main/ │ ├──
java/com/example/library/ │ │ ├── LibraryApplication.java │ │ ├──
config/DataInitializer.java │ │ ├── controller/LibraryController.java │ │ ├──
dto/BookAuthorDTO.java │ │ ├── entity/Author.java, Book.java │ │ ├──
repository/AuthorRepository.java, BookRepository.java │ │ └──
service/LibraryService.java │ ├── resources/application.properties │ └──
webapp/WEB-INF/views/ │ ├── add.jsp, list.jsp, update.jsp └──
test/java/com/example/library/ ├── BookRepositoryTest.java ├──
LibraryServiceTest.java └── LibraryApplicationTests.java
```

7. GitHub URL

<https://github.com/TejasKumat8/BooksAuth>

```
To push: bash git remote add origin
https://github.com/YOUR_USERNAME/YOUR_REPO_NAME.git git branch -M main git push -u
origin main
```